

Aarhus University

Project number - 2026F25

Parallax - Work Journal

Students:

Thea Teresa Lindgaard (202204350)
Niels Jakob Østerberg (202105879)
Kasper Stecher (202100451)
Kasper Bentsen (202103249)

Counsellor:

Lukas Esterle

Aarhus, May 29th 2026

Contents

1.	Split Screen POC	1
1.1.	Scene Setup	1
1.2.	Toggling Split Screen	2
1.3.	Caveats	2
2.	Player Controller POC	4
2.1.	Scene Setup	4
2.2.	Sphere Player Object	4
2.3.	Cube/Rectangle	5
2.4.	Two Player Objects	5
2.5.	Caveats	5
3.	Multiplayer POC	7
3.1.	Packages	7
3.2.	Setting the scene	7
3.3.	GUI	7
3.4.	Test	8
3.5.	Status	8
4.	Thirdperson Camera POC	9
4.1.	Requirements	9
4.2.	Prefab	9
4.3.	Scripts	9
4.3.1.	FollowCam	9
4.3.2.	PlayerMovement	11
5.	Asymmetric Perspective POC	13
5.1.	Defining Asymmetry	13
5.2.	Scene Setup	14
5.3.	Implementation	14
5.4.	Split screen support	16
5.5.	Result	17
6.	Character specific controller POC	19
6.1.	Scene Setup	19
7.	InputHandler	19
7.1.	BaseController	20
7.1.1.	HandleMovement()	20
7.2.	Cat Abilities	21
7.3.	Human Abilities	21
7.4.	Interface	21
8.	Tutorial POC	23
8.1.	Scene Setup	23
9.	Input Router	24
9.1.	Game Manager	25
10.	Ghost movement	26
11.	Objective & Tutorial Progression System	26
11.0.1.	Core Responsibilities	26

11.0.2. Ordered Checkpoint System	26
11.0.3. Progressive Visibility Control	27
11.0.4. Tutorial Instruction System	27
11.1. Checkpoint System	27
11.1.1. Responsibility and Separation of Concerns	28
11.1.2. Collector-Based Filtering	28
11.1.3. State Handling	29
12. Level Logic	30
12.1. Player Spawn Locations	30
12.2. Perspective Initialization	30

1. Split Screen POC

Author: Thea Teresa Lindgaard

A simple POC for split screen functionality was created. The purpose of the POC was to investigate the possibility of same-system multiplayer for manual testing of game features, despite the game concept not being compatible with split screen play. This will allow us to run the game with two players on the same computer, and thus removes the dependency on online multiplayer and networking features.

1.1. Scene Setup

In order to see the effects of a split screen, a simple scene was created. The scene contains two capsules in different colors representing each player on a simple plane, see **Figure 1**.

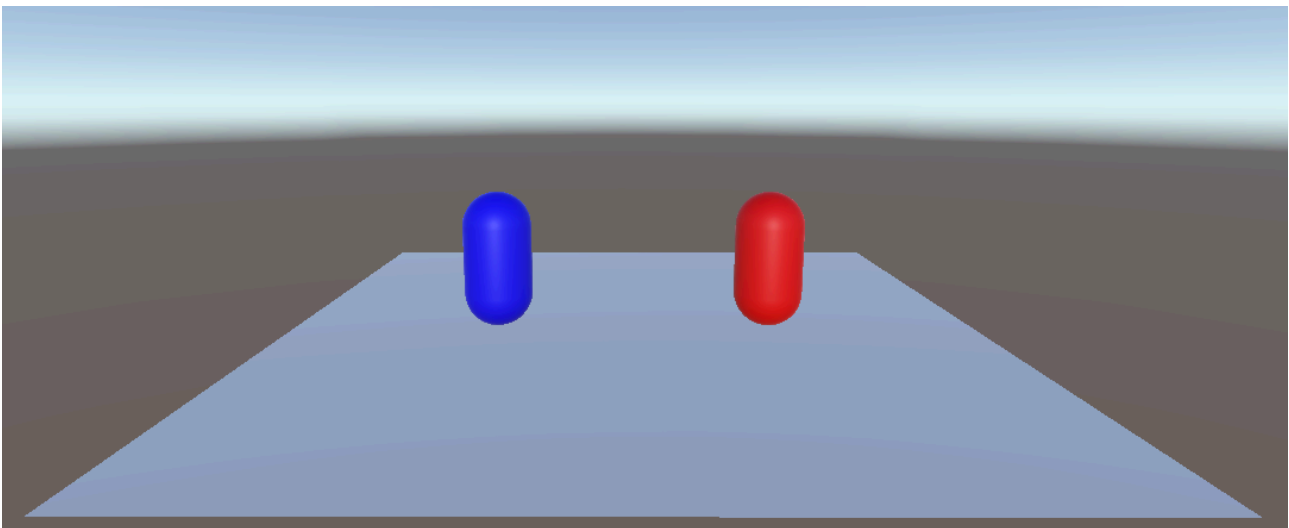


Figure 1: The scene view of the scene used in the split screen POC

Each capsule game object has a child object with a camera component, representing each player's unique FOV. By adjusting the cameras' viewport rects, we can achieve a setup where each camera is rendering on each half of the screen (**Figure 2**).

Viewport Rect	X	0	Y	0
	W	0.5	H	1
Viewport Rect	X	0.5	Y	0
	W	0.5	H	1

a) Viewport rect values for player 1 camera

b) Viewport rect values for player 2 camera

Figure 2: The values for each camera's viewport rect

This makes the player 1 camera occupy the left half of the screen, and the player 2 camera occupy the right half of the screen. In **Figure 3** you can see what it looks like in the game view.

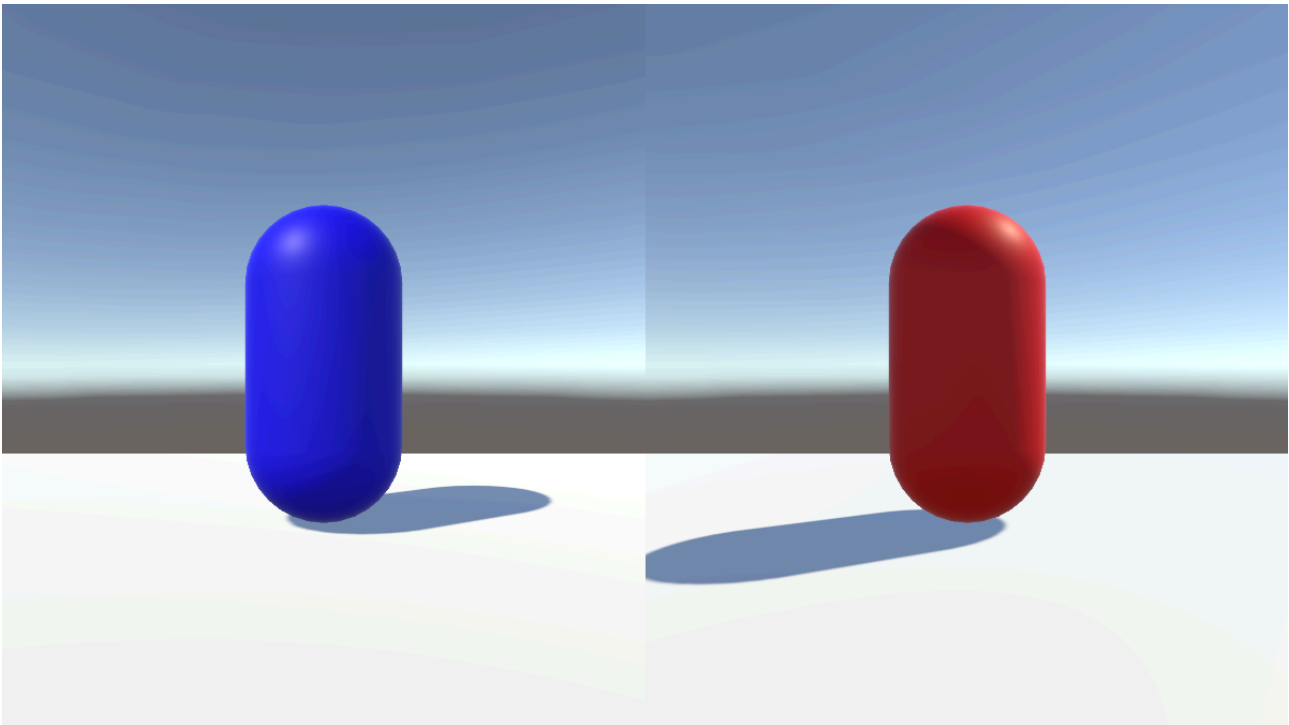


Figure 3: The game view of the scene used in the split screen POC, where each player has been rotated to look at each other

1.2. Toggling Split Screen

The split screen is useful for testing locally, when you need to see both players' FOV. But the game is not intended to be played on a split screen, as each player's perspective is supposed to be secret for the other player. Therefore, it was decided that a toggle function for the split screen could prove useful, as one might want to test the game from just one player's perspective.

For this, the F1 button is used as a toggle button for the split screen. When split screen is turned off, the viewport rect of the player 1 camera is adjusted to take up the whole screen, and the player 2 camera is disabled.

In order to capture F1 key presses, the Active Input Handling setting in Project Settings > Player > Other Settings have been set to "Both", although this is not strictly necessary, and can be changed later to use the newest Unity input system if so desired.

1.3. Caveats

The split screen can be useful for testing, but may also introduce additional complexity in regards to player controllers. The split screen will only be truly useful, if it is also possible to control both players on the same computer at the same time. This requires additional logic in the player controllers. The game needs to track whether the split screen is active, and if so, enable controls for both players.

Since the split screen is *only* needed for testing and debugging, controlling both players on the same computer should *also* only be possible in development. This means that the architecture must be designed in such a way, that couch co-op can be removed easily.

2. Player Controller POC

Author: Kasper Stecher

The purpose of this PoC was to experiment with how player movement can vary depending on character shape and physical properties. Two distinct placeholder shapes were used for testing: a Sphere and a Cube/Rectangle. Due to their fundamentally different physical characteristics, each shape was given a movement implementation that complemented its form. The shapes and mechanics implemented in this PoC are temporary and serve solely as experimental prototypes.

2.1. Scene Setup

In order to test this a simple scene with a ground object framed by walls was made, this allowed for both Player objects to be preset and move around.

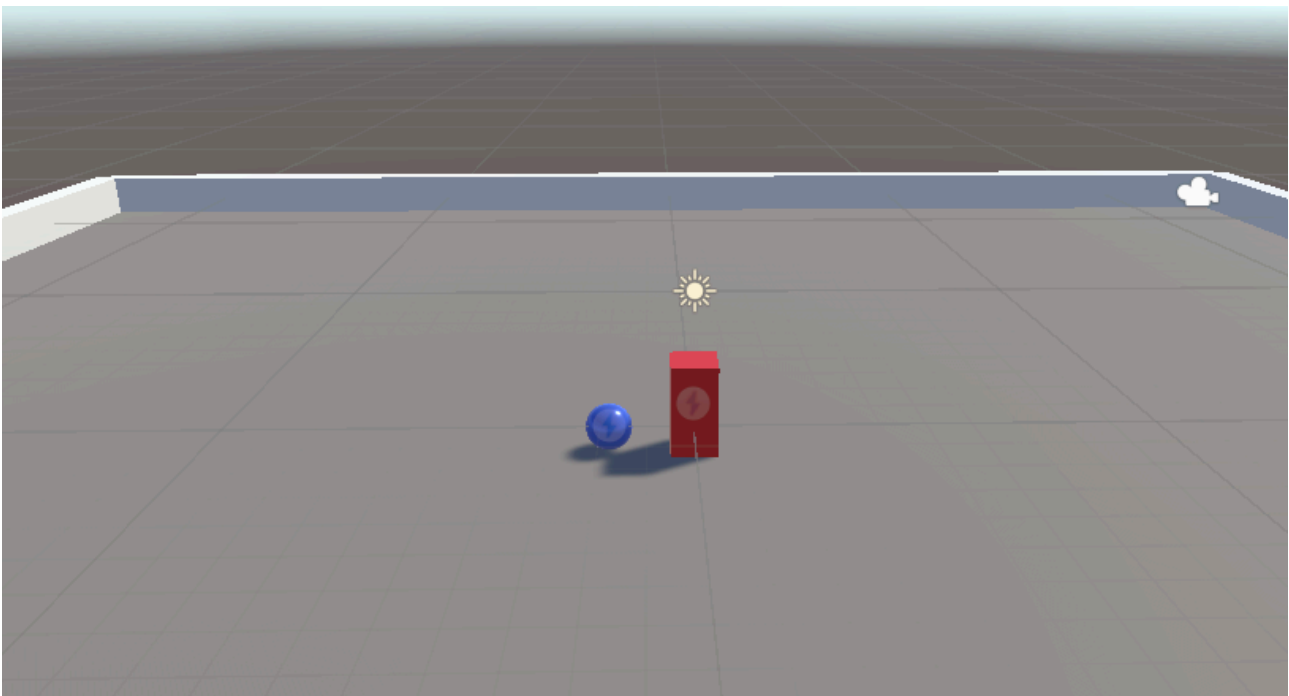


Figure 4: The scene view of the scene used in the player controller POC

Both player objects were configured with:

- A Rigid-body to enable physics-based interaction within the Unity engine.
- A custom material to visually differentiate the player types.

2.2. Sphere Player Object

Using Unity's Input System (Input Action Asset), movement across a flat plane was implemented.

To simulate a feeling of momentum, force was applied to the sphere based on its movement speed. Making the object feel less static and more physically grounded in the environment.

2.3. Cube/Rectangle

Using Unity's Input System (Input Action Asset), movement across a flat plane was implemented as for the sphere. To add the feel of a cube/rectangle, additional mechanics were introduced:

- While holding Shift, rotation constraints are temporarily disabled, allowing the object to tilt.
- Pressing R resets the object's rotation to its default upright state.

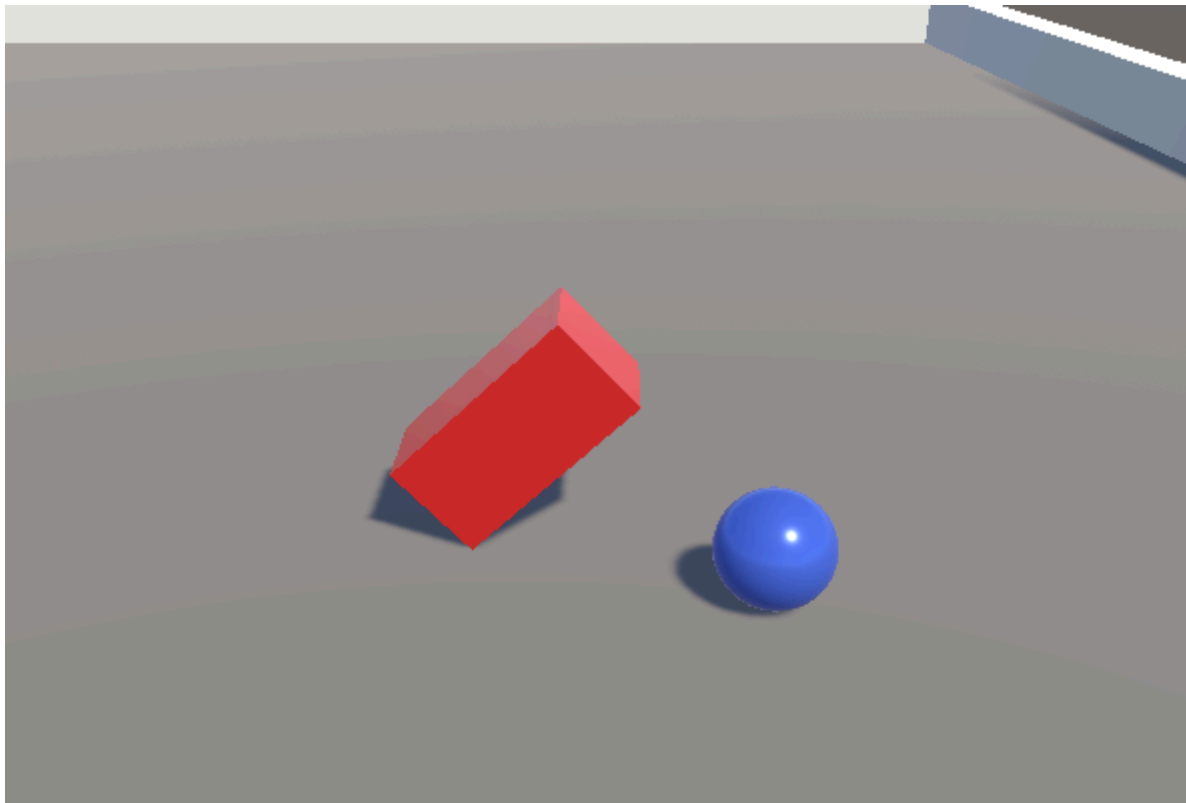


Figure 5: Showing the tilt motion of the Rectangle

2.4. Two Player Objects

In order to test movement and interaction between the two player objects each “player input” had to be set to a different “default Scheme” within Unity's Input System, for testing the Cube/rectangle was appointed “Keyboard&Mouse”, the sphere “Gamepad”.

2.5. Caveats

The ability to control both player objects simultaneously proved useful during development and testing, as it allowed rapid iteration without requiring multiple testers.

However, assigning fixed default control schemes (Keyboard & Mouse for the cube and Gamepad for the sphere) introduces potential limitations. In a final build, forcing specific input devices may lead to usability issues, particularly if players wish to configure their preferred input method. This will require further refinement, such as implementing dynamic device assignment or a player-join system.

At this stage, the selected player shapes (sphere and cube/rectangle) serve purely as Proof of Concept (PoC) representations. The final character designs have not yet been determined. As a result, mechanics such as the tilt functionality may become redundant or require redesign if the final player form does not support or benefit from such behavior.

<https://docs.unity3d.com/6000.3/Documentation/Manual/class-CharacterController.html>

3. Multiplayer POC

Author: Kasper Bentsen

The first step of the multiplayer POC is to create a Peer-to-peer (P2P) connection between two separate game objects that occupy the same virtual environment.

The goal of the POC is to create a small program where two instances of the game can be launched, one as a host and one as a client connected to the host, via local network. Each instance of the game should accurately reflect character movement and environment physics, synchronized between each of the players.

3.1. Packages

To enable P2P, the Unity package Netcode for GameObjects, is added to the project. The package includes scripts for network management and server hosting/joining.

3.2. Setting the scene

Following the documentation, an empty game object is created in the scene, named **NetworkManager**. For this to act as an actual network manager, the *NetworkManager* script from the **Netcode** package, is added to the object and configured by setting the transport protocol to **UnityTransport**.

To set the stage for the POC, a simple 3d plane is created as a reference point for the characters and a 3d capsule is created, to act as the character.

Since the character should be spawned whenever a player joins, it is dragged to a **Prefabs** folder, and deleted from the scene. The prefab is then added to the *Player Prefab* field in the network manager. A *NetworkObject* and *NetworkTransform* script, from the **Netcode** package is added to the prefab aswell as a simple movement script.

The movement script is given the type **NetworkBehaviour** and the player prefab is colored red for the owner, by validating against the **IsOwner** variable in the NetworkBehaviour class. On each update, the script validates if the owner of the prefab is trying to move it, and returns if it is not the owner.

3.3. GUI

Since the inspector can not be accessed from the built instance, a simple GUI is needed, to launch the host/client/server.

Another empty game object is added to the scene, with a script that renders three buttons, for starting the host, client and server respectively.

3.4. Test

To test the POC, the project is built and launched. The game is also run in the editor, to act as a client.

The host is launched in the built instance and the character is spawned and can be controlled with the W, A, S, D buttons, in accordance with the movement script **Figure 6**.

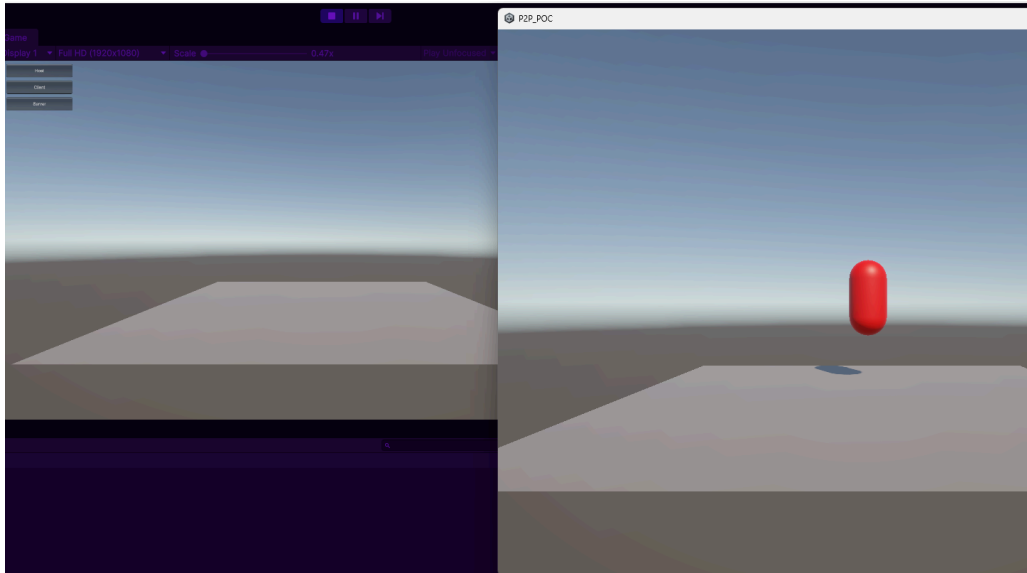


Figure 6: Host launched on the built game instance

The client is launched in the editor, and a character is spawned in both game instances **Figure 7**.

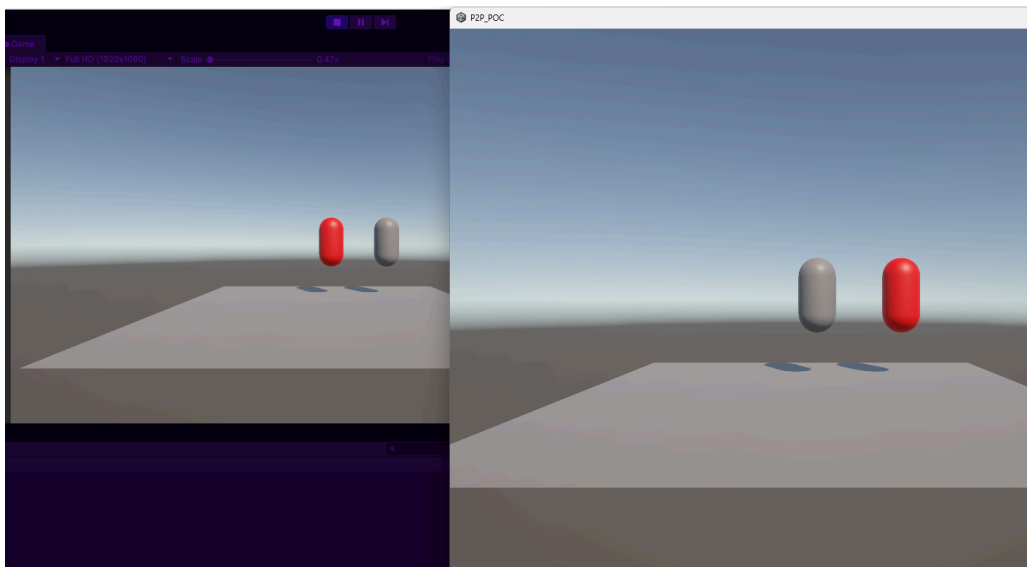


Figure 7: Client started and both characters visible and synchronized

Each game instance can only control the character whom they are the owner of.

3.5. Status

At this point the POC is finished. The next step is adding third person cameras to each character capsule, that follows the characters around.

4. Thirdperson Camera POC

Author: Kasper Bentsen

With the functioning P2P POC, the game needs a third person camera, that follows each player. This POC aims to develop a functioning third person camera prefab, that is instantiated and assigned to a player, as they invoke the *OnNetworkSpawn* method, in the *NetworkManager*.

4.1. Requirements

There are some simple requirements for the POC. The requirements aim to ensure that the POC is not too small and that the camera is actual functional in some way.

- 1) The camera should spawn with the player object
- 2) One camera per player
- 3) New players spawning should NOT transfer ownership of other cameras, from the already active players, to the newly spawned players
- 4) The camera should be rotatable
- 5) **(Optional)** The camera should have settings for FOV, distance and sensitivity
- 6) **(Optional)** The character movement should correspond with camera rotation

4.2. Prefab

The prefab is a simple camera prefab, with the Audio Listener removed. For the finished game, the audio listener would most likely be removed from the main camera instead, to keep the sole audio source on the character camera.

4.3. Scripts

The camera will need a *FollowCam* script, to work as intended. The script will contain info about the target for the camera, optional camera settings, the cameras position and methods to update the camera position, in respect to character movement and mouse rotation.

The *PlayerMovement* script will need to be updated to spawn the character when the player character spawns. It will also need a modification to it's movement logic, to facilitate movement, relative to the camera rotation.

4.3.1. FollowCam

The *FollowCam* script is a *MonoBehaviour* script with three components.

Local Variables

```
[CanBeNull] public Transform target;
```

```
public float distance = 4f;
```

```
public float sensitivity = 200f;
```

```

public float minY = -20f;
public float maxY = 60f;

private float currentX = 0f;
private float currentY = 20f;

```

The local variables specify the target for the camera and possible camera settings.

LateUpdate

```

private void LateUpdate()
{
    if (!target) return;

    HandleMouseInput();
    UpdateCameraPosition();
}

```

The *LateUpdate* checks if the camera has a target, and if it does, it handles mouse input and updates camera position.

Methods

```

void HandleMouseInput()
{
    if (Mouse.current.rightButton.isPressed)
    {
        Vector2 delta = Mouse.current.delta.ReadValue();

        currentX += delta.x * sensitivity * Time.deltaTime;
        currentY -= delta.y * sensitivity * Time.deltaTime;

        currentY = Mathf.Clamp(currentY, minY, maxY);
    }
}

```

The *HandleMouseInput* method, checks whether the right mouse button (RMB) is pressed, and reads the delta from the mouse input. It translates these to X and Y positions, and clamps Y between the set minimum and maximum values.

```

void UpdateCameraPosition()
{
    Quaternion rotation = Quaternion.Euler(currentY, currentX, 0);
    Vector3 direction = rotation * new Vector3(0, 0, -distance);

    transform.position = target.position + direction;
    transform.LookAt(target.position + Vector3.up * 1.5f);
}

```

The *UpdateCameraPosition* method handles camera rotation.

4.3.2. PlayerMovement

The player movement script needs to be updated to facilitate the camera features.

OnNetworkSpawn

The *OnNetworkSpawn* method needs to be overridden to assign the camera to the player character when it spawns on the network.

```
public override void OnNetworkSpawn()
{
    if (!IsOwner) return;

    spawnedCam = Instantiate(thirdPersonCameraPrefab);
    spawnedCam.GetComponent<FollowCam>().target = transform;

    base.OnNetworkSpawn();
}
```

As shown, the method is overridden to instantiate the camera prefab and then set the *FollowCam* scripts target, to be the current player.

Update

The *Update* method which was handling player movement before, needs to be modified to take the camera rotation into account when updating character movement.

```
void Update()
{
    if (!IsOwner) return;
    if (spawnedCam == null) return;

    Vector2 input = GetMovementInput();

    Vector3 camForward = spawnedCam.transform.forward;
    Vector3 camRight = spawnedCam.transform.right;

    camForward.y = 0;
    camRight.y = 0;

    camForward.Normalize();
    camRight.Normalize();

    Vector3 direction = (camForward * input.y + camRight * input.x).normalized;

    transform.Translate(direction * speed * Time.deltaTime, Space.World);

    if (direction != Vector3.zero)
```

```

{
    transform.rotation = Quaternion.LookRotation(direction);
}
}

```

The *Update* method has received a rather large overhaul, to only react if the camera has spawned and then using the cameras perspective to determine which direction to move.

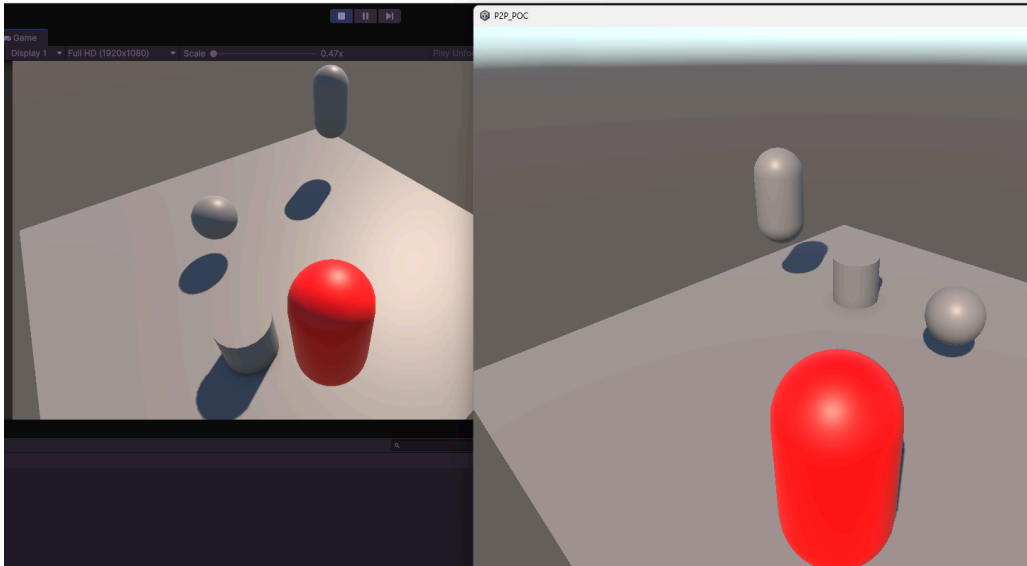


Figure 8: Both instances running with separate third person cameras

As shown in **Figure 8**, each instance of the game has their own third person camera assigned to the character.

5. Asymmetric Perspective POC

Author: Thea Teresa Lindgaard

The game concept revolves around asymmetric world views. For this reason, an investigation into the possibilities for implementing this was done.

5.1. Defining Asymmetry

Asymmetric perspectives can mean different things, and each thing can require a different implementation strategy, and provides their own puzzle-building possibilities. Some possible asymmetries include:

1) **Object Substitution**

- Each player sees an entirely unique version of an object, that may differ in size, shape and color, or one player doesn't see the object at all

2) **Light Asymmetry**

- Each player sees unique light levels or colors

3) **Collision Asymmetry**

- Each player collide (or don't collide) differently with objects in the scene

All of the above are related to an object's properties (color, collision, mesh etc.). In Unity, such properties are represented by components. By toggling these components, or configuring properties on the components, we can achieve player dependent asymmetry.

5.2. Scene Setup

To test how viable this approach is with a variety of different Unity components, I created a simple test scene. The scene contains objects and light sources (light not rendered in the picture below), as well as two capsules representing each player, a floor and some walls for the light to bounce off of.

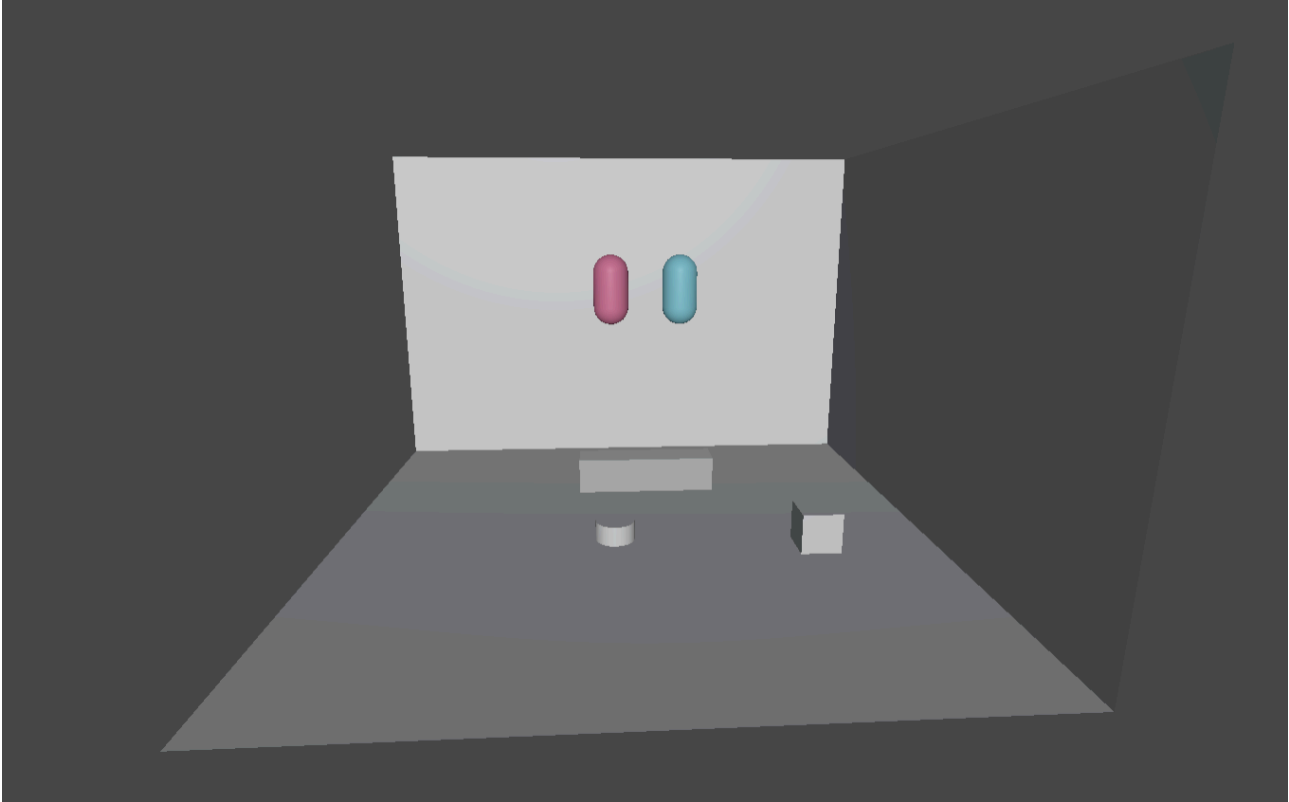


Figure 9: The scene view of the scene used to test

To make the scene completely dark, i navigated to Window > Rendering > Lighting > Environment, and configured the lighting as below:

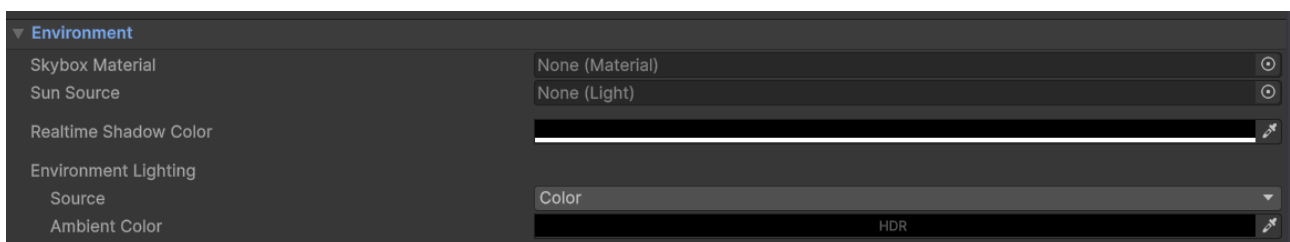


Figure 10: The environment light settings required to make a scene completely dark

5.3. Implementation

As many objects in a scene may need to be asymmetrical, it would be error-prone and inefficient to manage references to all asymmetrical objects manually. Instead, a base class for all asymmetrical properties was created. It simply contains the functionality to apply some perspective to an object.

```
public abstract class BaseAsymProperty : MonoBehaviour
{
```

```

    public abstract void ApplyPerspectiveProfile(PerspectiveProfile profile);
}

```

To automatically locate all of the scripts that derive from BaseAsymProperty in the scene in Unity, it must not be generic. As a workaround, the base class consists of two layers. The above, which defines the common method to apply the perspective, and the following, which is a specialization for some specific component T:

```

public abstract class BaseAsymProperty<T> : BaseAsymProperty where T : Component
{
    protected T AsymComponent;

    private void Awake()
    {
        AsymComponent = GetComponent<T>();
        if (AsymComponent == null)
        {
            Debug.LogError($"{gameObject.name} is missing {nameof(T)} component! Disabling script...");
            enabled = false;
        }
    }
}

```

This specialization also fetches the required component from the GameObject automatically.

A manager to apply the perspective to all asymmetrical objects is required. This class uses Unity's builtin method to find all BaseAsymProperty scripts in the scene. This is also the reason that a non-generic base class is required.

```
public class PerspectiveManager : MonoBehaviour
{
    private BaseAsymProperty[] _asymProperties;

    private void Start()
    {
        _asymProperties = FindObjectsByType<BaseAsymProperty>(FindObjectsSortMode.None);
    }

    public void ApplyPerspective(PerspectiveProfile profile)
    {
        foreach (BaseAsymProperty asymProperty in _asymProperties)
        {
            asymProperty.ApplyPerspectiveProfile(profile);
        }
    }
}
```

Finally, some concrete implementations for each desired type of asymmetry. Most of these enable or disable the component entirely, or switches out a material or mesh.

Collisions are a special case. Using Physics.IgnoreCollision allows us to inform Unity's physics engine to completely ignore collisions between two colliders. This requires a reference to the other collider. In my test scene, only the players have colliders that matter. To find them, i have assigned each player a tag, and i use that tag to fetch their colliders.

5.4. Split screen support

It would be advantageous to build the asymmetry such that it is compatible with split screen testing, but since split screen is just a tool, we do not want the architecture to be driven by split screen compatibility. Rather it would be preferable if the split screen could be added or removed at will, without requiring much work. Split screen would also be useful for testing the asymmetrical views, so using the same strategy as in the split screen POC, a simple split screen setup was achieved.

The split screen handling was modified slightly, so that one GameObject with the split screen tool script would be all that is required to enable split screen. This script manages spawning the half cameras, assigning them scripts, and toggling the perspectives just before each camera renders. Removing that GameObject disables split screen completely.

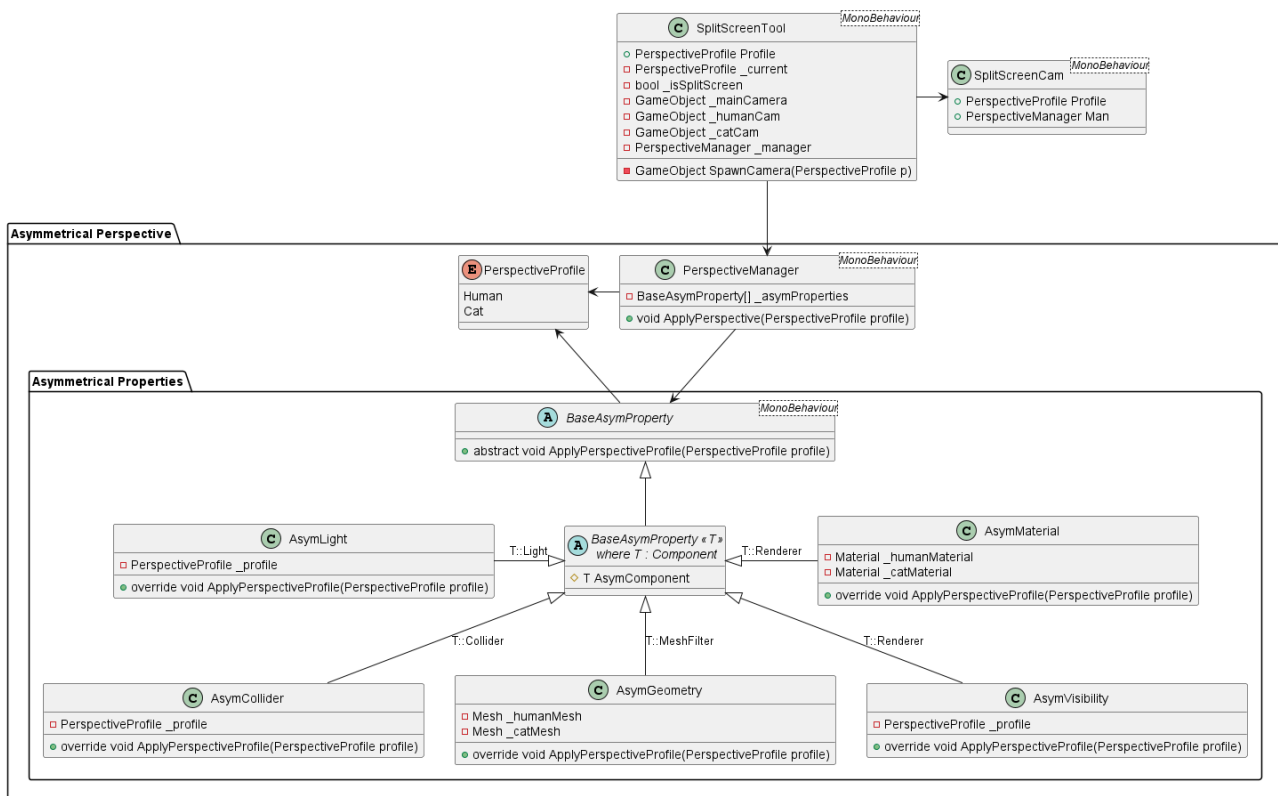


Figure 11: Class diagram over the code written for this POC

In the diagram, it can be seen how the split screen support is not an integrated part of the system, but rather something external that can be added and removed. The split screen manager hooks into the perspective manager to toggle perspectives before each render. This is inefficient, but offers the advantage that it can be removed.

5.5. Result

With everything in place, the final result of the asymmetry looks like this:

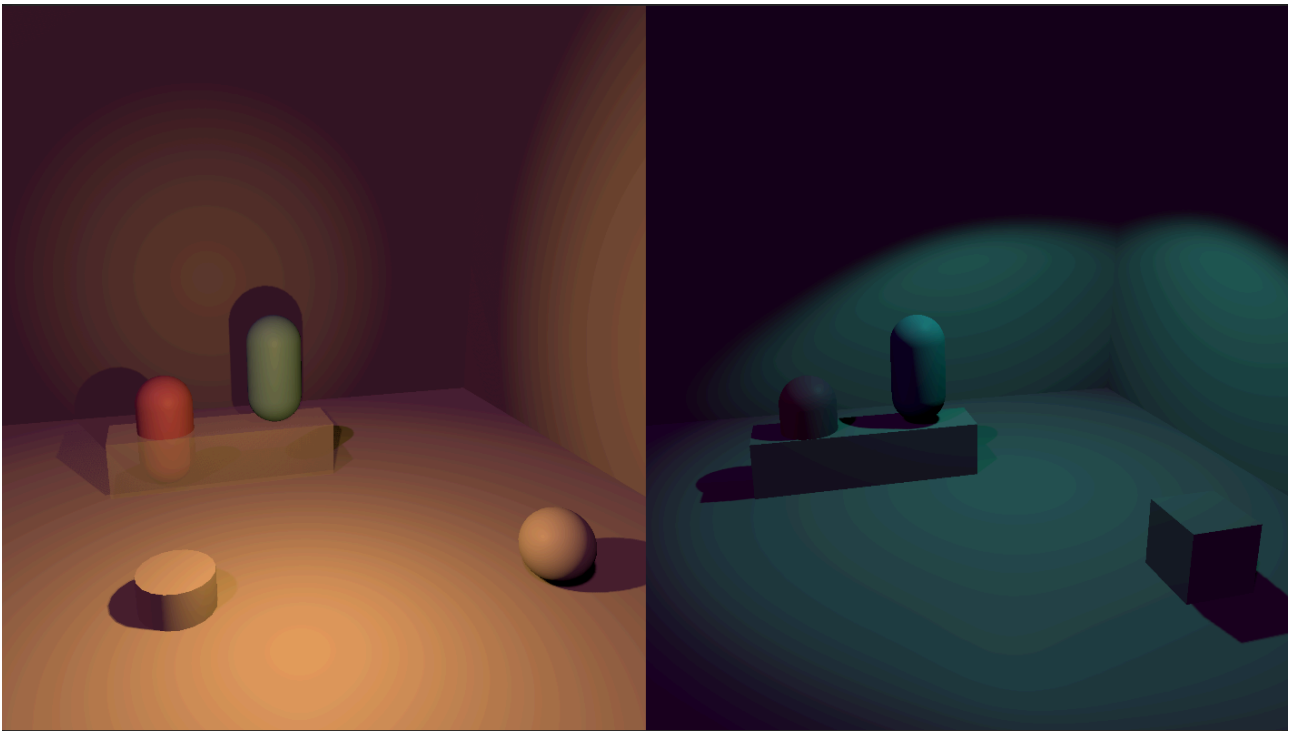


Figure 12: The split screen view of the different perspectives

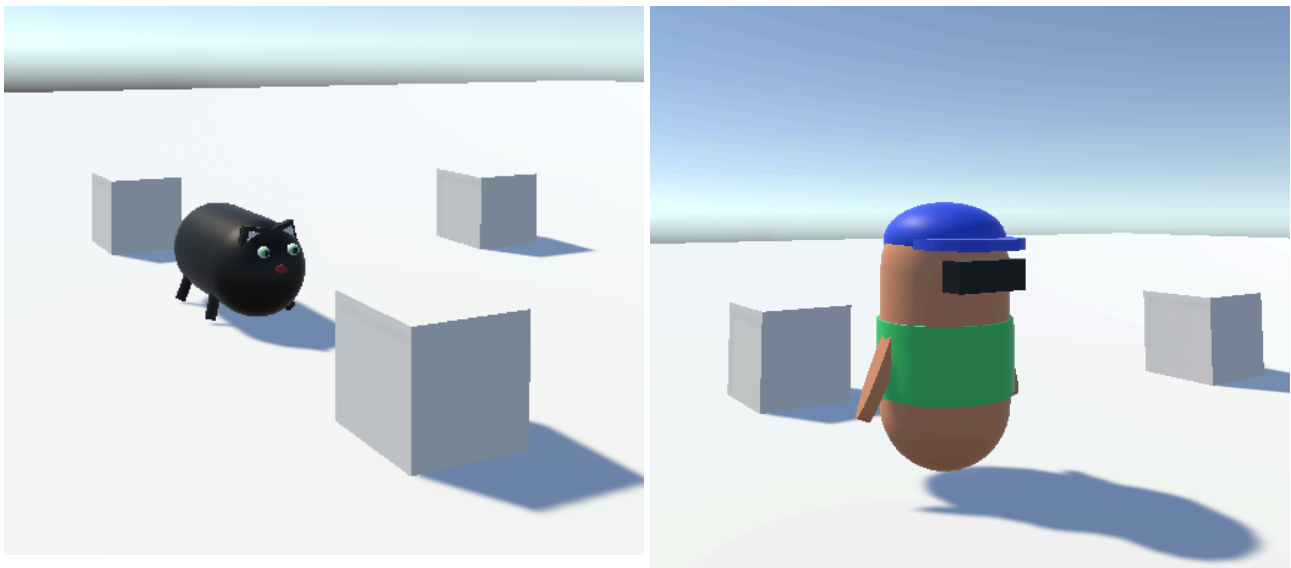
6. Character specific controller POC

Author: Kasper Stecher

The purpose of this PoC was to experiment with how player movement can be specific to the cat and human characters. To illustrate this a basic cat and human were made. These are not final designs merely a method to visually distinguish the two characters.

6.1. Scene Setup

In order to test this two simple scenes with a Plane object and a few obstacles to better show movement. The two scenes are to separately test the cat and human characters.



a) Cat in scene for testing movement

b) Human in scene for testing movement

Figure 13: The two temp. characters

7. InputHandler

The purpose of the InputHandler class is to act as an abstraction layer between Unity's Input System and the player character's gameplay logic. This ensures that input detection is separated from character behavior. Preventing too tightly coupled relation.

The implemented solution uses interfaces to decouple input detection from behavior.

```
private IMovement movement;  
private IJump jump;  
private ISprint sprint;
```

Each interface represents a capability:

- IMovement → movement and rotation
- IJump → jumping ability

- ISprint → sprinting ability

This allows different player types to implement only the capabilities they support.

7.1. BaseController

The BaseController implements a character movement system using Unity's built-in CharacterController. Its purpose is to handle movement, rotation, gravity, and vertical impulses (such as jumping).

The controller implements the IMovement interface, which decouples movement logic from input handling. This allows different player types (e.g., Cat and Human) to share the same movement controller while enabling different abilities, such as allowing only the Cat to jump and human to sprint.

The different movement parameters are exposed in the Unity Inspector to allow character-specific configuration depending on gameplay requirements and desired movement feel.

```
[Header("Movement")]
[SerializeField] [Range(0f, 20f)] public float moveSpeed = 5f;
[SerializeField] [Range(0f, 36f)] private float rotationSpeed = 18f;

[Header("Acceleration")]
[SerializeField] [Range(0f, 10f)] private float acceleration = 5f;
[SerializeField] [Range(0f, 10f)] private float deceleration = 5f;
[SerializeField] [Range(0f, 10f)] private float airControl = 0.5f;

[Header("Gravity")]
[SerializeField] [Range(-50f, 0f)] private float gravity = -20f;
[SerializeField] [Range(-10f, 0f)] private float groundStick = -2f;
```

7.1.1. HandleMovement()

The HandleMovement() method calculates player movement relative to the camera's horizontal rotation (yaw), ensuring intuitive third-person controls. The desired movement direction is derived from player input and camera orientation, allowing forward movement to align with the camera view.

```
Quaternion yawRot = Quaternion.Euler(0f, camYaw, 0f);

Vector3 wishDirection = camForward * moveInput.y + camRight * moveInput.x;
```

Movement velocity is calculated based on acceleration, deceleration, and air control, allowing different behaviour when grounded versus airborne:

```
Vector3 targetVelocity = wishDirection * moveSpeed * speedMultiplier * control;

currentHorizontalVelocity = Vector3.MoveTowards(
    currentHorizontalVelocity,
    targetVelocity,
```

```
        accelRate * Time.deltaTime
    );
```

Finally, horizontal movement and vertical velocity (gravity or jump) are combined and applied using Unity's CharacterController:

```
Vector3 motion = currentHorizontalVelocity + Vector3.up * verticalVelocity;
cc.Move(motion * Time.deltaTime);
```

7.2. Cat Abilities

The CatAbilities component implements the IJump interface and provides jump functionality for the Cat character. It applies a vertical impulse to the BaseController when the jump action is triggered, but only if the character is grounded, ensuring realistic jump behaviour and preventing mid-air jumping.

```
if (!cc.isGrounded) return;
```

Applying the jump impulse via the movement controller:

```
controller.AddVerticalImpulse(jumpImpulse);
```

This design separates abilities from the core movement controller, allowing different characters to have different abilities while sharing the same movement system.

7.3. Human Abilities

The HumanAbilities component implements the ISprint interface and provides sprint functionality by modifying the movement speed multiplier in the movement controller. This allows the Human character to temporarily increase movement speed while sprinting.

```
locomotion.SetSpeedMultiplier(sprinting ? sprintMultiplier : 1f);
```

This approach separates character abilities from the core movement system, allowing different characters to have unique abilities while reusing the same locomotion controller.

7.4. Interface

The interfaces define a abstraction layer between input handling and character behaviour. This allows different characters to implement only the abilities they support, improving flexibility and separation of concerns.

```
public interface IMovement
{
    void Move(Vector2 move);
    void Rotate(Vector2 look);
}
```



```
public interface IJump
{
    void Jump();
}

public interface ISprint
{
    void SetSprinting(bool sprinting);
}
```

The IMovement interface defines basic movement functionality, while IJump and ISprint define optional abilities.

8. Tutorial POC

Author: Kasper Stecher

The purpose of this PoC was to experiment with setting up a tutorial environment where all the necessary components are available to later use for the actual tutorial.

8.1. Scene Setup

To support development and testing of the tutorial proof of concept, a dedicated test scene was created in Unity.

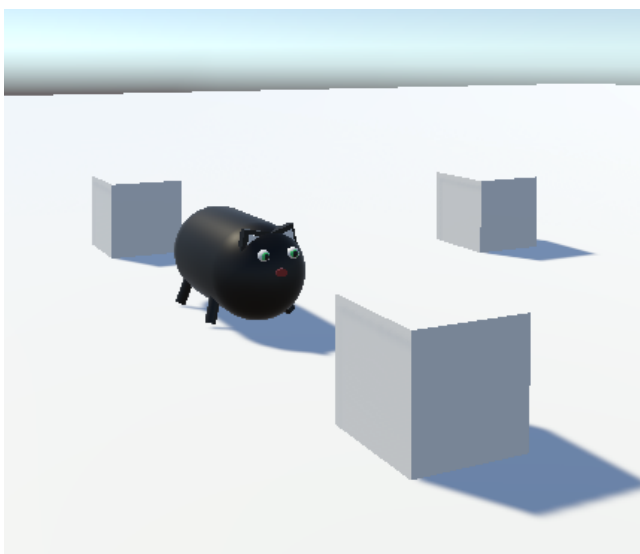
The scene consists of a flat ground plane acting as the base surface with a simple room-like structure defining the playable boundaries.

Inside the building are multiple blocks and elevated ledges of varying heights to test:

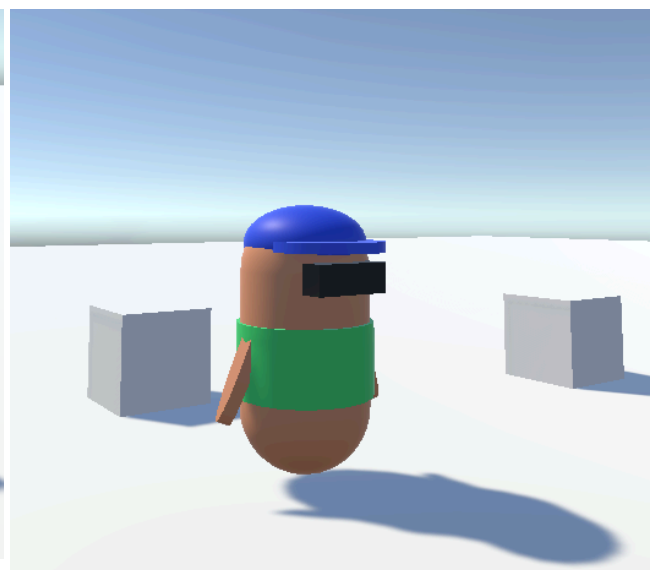
- Horizontal movement
- Vertical navigation (jump mechanics)
- Collision handling
- Two interactable pushable boxes to test physics-based interaction

Two player objects to validate character switching and input routing.

The purpose of this setup was not to create a final level, but rather a controlled environment where core mechanics could be tested in isolation, including movement, interaction, checkpoint progression, and tutorial flow control.



a) Cat in scene for testing movement



b) Human in scene for testing movement

Figure 14: The two temp. characters

9. Input Router

To support dynamic player switching in the tutorial prototype, an input abstraction layer was introduced in the form of an InputRouter.

Instead of binding input directly to individual player objects, a centralized routing component was implemented. This component forwards input events to whichever player is currently active.

```
public class InputRouter : MonoBehaviour
{
    private IMovement movement;
    private IJump jump;

    public void SetActive(GameObject playerRoot)
    {
        movement = playerRoot.GetComponent<IMovement>();
        jump      = playerRoot.GetComponent<IJump>();
    }

    public void OnMove(InputValue value)
        => movement?.Move(value.Get<Vector2>());

    public void OnJump(InputValue _)
        => jump?.Jump();
}
```

The main purpose of the InputRouter is to decouple:

Input handling

Player-specific movement logic

Instead of enabling and disabling multiple PlayerInput components when switching characters, input events are received once and dynamically redirected to the currently active player.

The router does not depend on specific player classes. Instead, it relies on the interfaces:

- IMovement
- IJump

This follows the Dependency Inversion Principle (SOLID), where high-level modules (input system) do not depend on low-level concrete implementations (e.g., CatController or HumanController).

As long as a player implements the required interfaces, it can receive input through the router.

9.1. Game Manager

The GameManager is responsible for high-level game flow control within the tutorial prototype. Its primary responsibility is to manage which player is currently active and ensure that all related systems (input routing and camera tracking) are synchronized accordingly.

```
public void OnSwitch(InputValue value)
{
    if (!value.isPressed) return;

    Debug.Log("R pressed → switching player");

    if (active == PlayerA)
        SetActive(PlayerB);
    else
        SetActive(PlayerA);
}
```

When the switch input is triggered:

- 1) The active player is determined.
- 2) The alternative player is selected.
- 3) SetActive() is called.

```
private void SetActive(GameObject player)
{
    activeMovement?.Move(Vector2.zero);

    active = player;

    activeMovement = active.GetComponent<IMovement>();
    inputRouter.SetActive(active);

    if (followCam) followCam.SetTarget(active.transform, snapToDefaultAngles: false);

    Debug.Log("Active player is now: " + active.name);
}
```

- 1) InputRouter and camera are updated.

This setup separates:

- Input detection (OnSwitch)
- State transition logic (SetActive)

This improves readability and maintainability.

10. Ghost movement

An issue during development was continued movement when switching players while holding directional input.

To mitigate this, the following line was introduced:

```
activeMovement?.Move(Vector2.zero);
```

This forces the previously active player to stop immediately before control is transferred.

Although functional, this solution is acknowledged as a temporary architectural compromise. A more robust design would introduce a `Stop()` method in the `IMovement` interface to explicitly reset internal movement state.

11. Objective & Tutorial Progression System

The `ObjectiveManager` is responsible for controlling tutorial progression and checkpoint sequencing within the proof of concept.

Rather than allowing players to activate checkpoints all at once, the system enforces a structured order. This ensures that the tutorial unfolds step-by-step.

11.0.1. Core Responsibilities

The `ObjectiveManager` handles:

- Sequential checkpoint activation
- Progress tracking
- Tutorial instruction messaging
- Completion detection and termination

It acts as the central hub for tutorial flow logic.

11.0.2. Ordered Checkpoint System

Checkpoints are manually assigned in the inspector:

```
[SerializeField] private List<Checkpoint> checkpoints;
```

This allows a specific ordering of tutorial objectives.

When a checkpoint is collected, the manager verifies:

```
bool isCurrentlyVisible =  
    checkpoints.Contains(checkpoint) &&  
    checkpoints.IndexOf(checkpoint) >= currentIndex &&  
    checkpoints.IndexOf(checkpoint) < currentIndex + showNextCount;
```

This ensures that only the currently “active” checkpoints can be collected, and players cannot skip ahead.

11.0.3. Progressive Visibility Control

The showNextCount variable determines how many upcoming checkpoints are visible at once.

```
[SerializeField, Min(1)] private int showNextCount = 1;
```

Visibility is applied through:

```
private void ApplyVisibility()
```

This allows for different approaches to the tutorial progression:

- Linear progression (showNextCount = 1)
- Semi-open progression (multiple visible objectives)

This design made it easy to experiment with tutorial pacing during development.

11.0.4. Tutorial Instruction System

Each checkpoint triggers feedback:

```
switch (collectedCheckpoints)
{
    case 1:
        StatusUI.Instance.Show("Maybe we need to use the blue to push the box?...");
        break;
    ...
}
```

instructional prompts are embedded within progression milestones.

11.1. Checkpoint System

The Checkpoint component represents an individual objective node within the tutorial system.

Each checkpoint is responsible only for detecting valid player interaction and delegating progression control to the ObjectiveManager.

```
public class Checkpoint : MonoBehaviour
{
    [Header("Who can collect this checkpoint?")]
    [SerializeField] private CollectorType requiredType;

    public CollectorType RequiredType => requiredType;

    public bool IsCollected { get; private set; }
}
```

```

public bool IsActive => !IsCollected;

private void OnTriggerEnter(Collider other)
{
    if (IsCollected) return;

    var identity = other.transform.root.GetComponent<CollectorIdentity>();
    if (identity == null) return;

    if (identity.collectorType != requiredType) return;

    if (ObjectiveManager.Instance != null)
    {
        ObjectiveManager.Instance.CollectCheckpoint(this);
    }
}

public void MarkCollected()
{
    IsCollected = true;
    gameObject.SetActive(false);
}

public void SetActive(bool active)
{
    gameObject.SetActive(active && !IsCollected);
}
}

```

11.1.1. Responsibility and Separation of Concerns

The Checkpoint class follows a single-responsibility design:

It is responsible for:

- Detecting trigger interaction
- Validating collector eligibility
- Informing the ObjectiveManager

It is not responsible for handling any progression state or any game state at all it simply informs the objectiveManager.

11.1.2. Collector-Based Filtering

Each checkpoint defines a required collector type:

```
[SerializeField] private CollectorType requiredType;
```

When a player enters the trigger, the system checks:

- Whether the object has a CollectorIdentity
- Whether its collectorType matches the required type

```
public enum CollectorType
{
    Blue,
    Green,
    Box
}
```

This allows for character-specific objectives.

This was particularly important in the tutorial PoC, where mechanics are introduced gradually through role-specific tasks.

11.1.3. State Handling

Each checkpoint maintains minimal internal state:

- IsCollected
- Active/inactive state

Collection results in:

```
IsCollected = true;
gameObject.SetActive(false);
```

Visibility is externally controlled via:

```
SetActive(bool active)
```

This allows the ObjectiveManager handle the logic regarding visibility and current state.

12. Level Logic

Author: Thea Teresa Lindgaard

As part of real implementation work, some level logic was implemented. Level logic covers basic level flow control, such as initializing the level post scene-load, handling win/loss condition checks, and restarting or ending the level.

To manage the level flow a LevelManager class was created. An instance of this manager will exist in each level, and perform initialization in Unity's builtin Awake and Start methods, thus decoupling it entirely from other application flow control systems.

12.1. Player Spawn Locations

As the player objects exists from the moment a player joins the multiplayer sessions, we needed a way to move players into position once a level has loaded. This is managed by the level manager. The spawn locations are manually configured in the editor, as they are dependent on the level layout. The manager fetches the players, distinguished by their tags, and moves them to the pre-configured spawn positions.

12.2. Perspective Initialization

The Network Manager provides a method with which we can find the local player object. Using this, we can find the tag of that player, and perform initialization base on that tag. The perspective manager handles this itself, and the level manager simply triggers this initialization.